BDD Test Generator

Handbook — setup guide and full reference. Writes BDD test cases in Gherkin from a link to a page or a plain description, then lets you review, revise and export them. Runs entirely on your own machine.

Part 1 — Setup guide

Start here. Everything you need to install it and generate your first test cases.

Writes BDD test cases (Gherkin) from a link to a page, or from a plain description. Runs entirely on your own machine.

What you need first

Node.js 18+	**nodejs.org — take the LTS installer**
Claude Code, signed in	claude.com/claude-code. After installing, open a terminal, run `claude`, and sign in once. Then close it.

You do **not** need an API key. The app uses whatever Claude account you signed into Claude Code with.

Setup (2 minutes)

1. Unzip the folder anywhere.
2. Double-click `start.bat`.

That's it. The first run installs dependencies (about a minute), then opens the app at **http://localhost:4173** in your browser.

A second window opens and stays open — that's the server. **Leave it running.** Close it when you're done.

Prefer the terminal?

```
cd TestGenerator
npm install
npm start
```

Your first test cases

1. In the box, paste **either**:

- a plain description — *"Users reset their password via an emailed link. The link expires after 1 hour."* -> click **Generate test cases** directly.
- a bare link to a page — `https://your-app.example.com/login` -> click **Explore the whole site first** instead (a link on its own always goes through this; Generate stays disabled for it). Add a sentence of description alongside the link and Generate opens back up, using that text as context.

1. Watch the **Live activity** panel — it shows the browser opening the page, reading it, and writing.

A description takes under a minute. Exploring a site is usually 15–25 seconds; generating from the pages you pick is 1–3 minutes per page.

Then in **Test Cases** you can edit any step, tick/untick what to keep, write a review note on a scenario and press **Update** to have it rewritten, or use **Update all** to apply one note across everything. **Download** gives you `.feature`, Excel, a traceability matrix, or a share file.

Common things

Explore a whole site instead of one page Click **Explore the whole site first**. It lists the pages it finds, you tick the ones you want, then generate. Budget roughly 3 minutes per page.

Pages behind a login Two ways in. If you have a **test account**, the fastest is **Options -> Test username / password** plus ticking **"Let it interact with the page"** — the agent signs in with exactly that pair and nothing else. No test account? Open **Options -> Borrow your Chrome session** and follow the three steps — you copy a snippet, run it in the Chrome tab you're *already* signed into, and paste the result back. Your password never leaves your browser either way — you never type it into this app, only into the real site or the dedicated field the agent alone uses.

> Session-borrowing doesn't work on every site: browsers hide `HttpOnly` cookies from scripts. The app tells you what it captured. If the site uses those, run `npm run login` instead and sign in through the window it opens.

Give it more to go on Options -> Extra instructions — a free-text box for what the product is or which flow to focus on. **Options -> Upload product info** accepts a `.txt`, `.md`, `.pdf`, or `.docx` spec and drops its text into that same box, labeled with the filename, so you can see and trim exactly what gets sent.

Change what kinds of test it writes Options -> Kinds of test to write. Happy path, negative, validation and edge cases are on by default. Permissions, API, performance, security, accessibility and data integrity are opt-in.

Someone sent you a `.feature` or `.testrun.json` Options -> Import a file someone sent you. Drop it in — it becomes a new run you can review and re-export. It won't overwrite anything you already have.

If something goes wrong

Symptom	Fix
`'npm' is not recognized`	Node.js isn't installed, or you need to reopen the terminal after installing it.
Warning about the `claude` CLI	Claude Code isn't installed or isn't signed in. Run `claude` once in a terminal and sign in.
"A server is already running"	It's already up — the script just opens your browser. To restart, close the server window first.
Generation fails immediately	Almost always Claude Code not being signed in. Run `claude` in a terminal to check.
Times out on a URL	Pick fewer pages, or paste a description instead. Each page costs a page-load and a read.
Blank page at localhost:4173	Give the server window a few seconds, then refresh.

Run `npm test` any time to check the app's own logic is intact (78 tests, no network needed).

Before you share this folder onward

Double-click `package-for-sharing.bat`**.**

It builds `TestGenerator-share.zip` next to the folder (~127 KB) with your own data stripped out, so you can't accidentally ship a live login session. Send that file.

What it leaves out, if you'd rather zip it by hand

```
node_modules/     51 MB, rebuilt by npm install
data/             your runs — and a saved login session, if you have one
.browser-profile/ a saved login session
.playwright-mcp/  scratch files
.env
mcp/playwright-profile.generated.json    (has your Windows username in a path)
```

> `data/` **and** `.browser-profile/` **can hold a working session for a site you signed into.** Anyone who gets those files gets that session. This is the one thing worth double-checking.

A few honest limits

- Test cases are a **strong first draft**, not a finished suite. Read them before they go into your tracker.
- It won't invent error messages it hasn't seen. If it couldn't observe the exact wording, it asserts the behaviour instead (*"an error message is shown"*) — that's deliberate.

- Whatever you paste, and any page you point it at, is processed by Claude. **Don't paste real customer data, credentials, or production PII.** Use staging with synthetic data. On a regulated project, check with your compliance team first.

Part 2 — Full reference

How each part works, and the details behind the options.

A local, shareable framework that generates BDD test cases (Gherkin) from a **link to a live page/UI** or a plain requirement/description — written the way a senior QA lead with 20+ years of experience would write them: natural, simple English that any stakeholder can read. It opens a local review site where you can edit, remove, or tick/untick each step, leave a review note on any test case and have it revised automatically, and track requirement coverage in a separate tab. You can then download the result as a `.feature` file.

Generation runs through your **local Claude Code CLI** (`claude`), not a separate API key — it uses whatever account (Pro/Max subscription or API key) you're already logged into Claude Code with on this machine.

Requirements

- Node.js 18+
- Claude Code installed and logged in on this machine (run `claude` once interactively first if you haven't) — the `claude` binary must be on your PATH.
- For URL-based generation: nothing extra to install. The app launches Playwright MCP on demand via `npx` (first run downloads it, which takes a minute).

What's in this folder

```
server/            Express backend (API + static file serving)
  lib/
    prompts.js       The QA-lead persona and generation/revision prompts
    claudeCli.js      Shells out to the local `claude` CLI (headless/-p mode) for
                       generation and review-driven revision. Auto-detects whether
                       the input is a URL; if so it drives a real browser through
                       Playwright MCP (navigate + accessibility snapshot) and
                       streams progress back as it works
    crawler.js        Fast site discovery: a real headless browser via playwright-core
                       directly, no model call — same-origin BFS over <a href> links
    docExtract.js     Text extraction from an uploaded product-info file
                       (txt/md via Buffer, pdf via pdf-parse, docx via mammoth)
    urls.js           URL normalization shared by both discovery paths, so a page
                       counts as "the same page" identically either way
    placeholders.js   Scenario Outline placeholders: extraction, rename-sync, validation
    revise.js         Tags steps [REVISE]/[KEEP] for the prompt and reconciles the
                       model's reply, guaranteeing unticked steps survive verbatim
    testTypes.js      The 10 test types: prompt guidance, Gherkin tag, defaults
    quality.js        Deterministic checks — duplicates, coverage gaps, plain-English lint
    cucumberChecks.js Cucumber-specific: Given/When/Then grammar, step smells,
                       voice consistency, and the step-reuse metric
    versions.js       Snapshots before every change, plus the diff engine
    gherkin.js        .feature text and aligned data-table rendering
    featureParser.js  Parses a .feature file back in (import), incl. outlines and tables
    excel.js          Workbook builder (Summary / Test Cases / Traceability)
    store.js          Saves/loads each generation run as a JSON file in data/runs
    session.js        Borrowing a signed-in session from your own browser: the
                       console snippet, and conversion to a Playwright storageState
    audit.js          Append-only audit log of actions (data/audit.log)
  routes/            generate (+ discover: crawl a site and list its pages),
                     runs (save/update-one/update-all/fill-gaps/versions),
                     export (.feature/.xlsx/.json share), import, productInfo
mcp/playwright.json  Playwright MCP definition (headless, throwaway profile - the default)
scripts/login.js     One-time manual sign-in for pages behind a login (npm run login)
.browser-profile/    Created by npm run login: a saved browser session (gitignored,
                     holds live cookies - never share this folder)
test/run-tests.js    Offline suite for the deterministic logic (npm test)
public/              The review UI (plain HTML/CSS/JS, no build step)
skills/gherkins/     The underlying Claude Code skill this framework is based on —
                     kept here so the authoring rules travel with the app and can
                     also be used directly inside Claude Code (`/gherkins`).
data/                Created at runtime: one JSON file per generation run + audit.log
                     (gitignored — this is where requirement text ends up)
```

Setup

Easiest: double-click start.bat — it installs dependencies on first run, starts the server in its own window, and opens the app in your browser automatically.

Or manually:

```
npm install
npm start
```

Open the URL it prints (default `http://localhost:4173`).

Using it

Choosing what kinds of test to write

Before generating, tick the kinds of test you want. Four are on by default — **Happy path**, **Negative path**, **Field validation**, **Edge cases**. Six more are opt-in: **Permissions & roles**, **Backend / API**, **Performance**, **Security**, **Accessibility**, **Data integrity**.

Each ticked type becomes an instruction the generator must satisfy *and* a Gherkin tag on the resulting scenarios (`@happy-path`, `@security`, …), so your CI can filter on them. If you ask for a type and nothing comes back tagged with it, the quality check tells you.

The review pass

Review the draft before showing it is on by default. After the first draft, a second pass looks at it fresh and returns corrections only — merge these two duplicates, add a scenario for that uncovered requirement, rewrite this step that describes clicking a button. What it changed is listed under *What the review pass changed*, so nothing is silently rewritten.

It roughly doubles generation time (~3 minutes for a big suite). Untick it if you want the raw draft fast.

Extra instructions, test credentials, and product-info files

Three optional fields in **Options**, all read as context rather than acted on (except the credentials, see below):

- **Extra instructions** — free text: what the product is, which flow to focus on, anything worth knowing that isn't visible on the page itself.
- **Test username / password** — a dedicated pair of fields, kept separate from the free-text box on purpose. With **"Let it interact with the page"** also on, the agent signs in using *exactly* this pair — never a credential it happens to see published on the page itself, and it's instructed never to repeat the literal values anywhere in its output. As a backstop, the server also strips any exact match of the password (and username) from the saved run before it's written to disk or sent back to the browser, so even a slip-up can't leak it into an exported file. With interaction off, credentials aren't used or sent at all.
- **Upload product info** — drop a `.txt`, `.md`, `.pdf`, or `.docx` file (a spec, a PRD, a set of notes) and its text is extracted and appended into the Extra instructions box, labeled with the filename, so you can see and edit exactly what will be sent before generating. PDF/DOCX extraction uses `pdf-parse` and `mammoth`; a scanned PDF with no real text layer will extract nothing and say so rather than fail silently.

Quality checks

These run on every generation, save, and update — no AI call, instant, and they report in the **Quality check** bar:

Check	What it catches
Near-duplicate scenarios	Two test cases that verify the same thing, by comparing step-sequence and overall word overlap

Check	What it catches
Uncovered requirements	A requirement no ticked scenario maps to
Missing requested types	You asked for Security tests and none exist
Scenario Outline errors	A `<placeholder>` with no matching Examples column (Cucumber would fail), an unused column, placeholders outside an outline, or an outline with no rows
Cucumber grammar	A scenario with no Then, a Then before its When, setup after the action, or more than one When block (two behaviours in one scenario)
Step smells	`should`/`will`/`must` in a Then, two actions joined by "and", "verify"/"check" inside a step, or a vague assertion ("works correctly")
Voice consistency	The suite mixing "user"/"customer"/"visitor" for one actor, or first person with third
Step reuse	Near-verbatim step pairs that would each need their own step definition, plus a **reuse metric**: total steps, distinct step definitions required, and the ratio
Plain-English lint	Steps that drifted into UI mechanics (`clicks`, selectors, HTTP verbs, `shall`), hardcoded URLs, or ran past ~22 words

The deterministic checks and the AI review pass catch different things: the checks catch near-verbatim repeats reliably, the review pass catches *semantic* duplicates (two scenarios differing only by input value) that word-overlap can't see.

1. **Generate** — paste either:

- a **URL** to a live page (e.g. a staging login or form page) — the app opens it in a real headless browser, takes an accessibility snapshot, and grounds the scenarios in the page's actual field labels, button text, links, and headings, or
- a **plain description, user story, or acceptance criteria**.

Optionally tick **"Let it interact with the page"** (URLs only) to let it type into fields and submit forms so it can capture the real validation message wording. Off by default — see **Interactive exploration** before using it.

Click *Generate test cases*. A **Live activity** panel shows what the agent is doing as it works (opening the page, reading the structure, typing, writing the test cases). A live page usually takes 1–3 minutes.

1. **Review, in the Test Cases tab** — for each scenario you can:

- tick/untick the scenario or any individual step (see **What the tick controls**)
- edit the title or step text directly, and change any step's keyword (Given/When/Then/And/But) from its dropdown
- remove a step, or add a new one with whichever keyword you need
- edit any **data table** the generator attached to a step — cells, rows, columns — exported as an aligned Gherkin pipe table
- turn a word into a `<variable>`: select it in the step and press <> (see **Scenario Outlines**)
- leave a review note (e.g. *"add a step for an expired link"*) and click **Update** — Claude revises just that scenario to address the note

- delete a scenario entirely, or add a blank one
- click **Save changes** to persist edits, or **Export .feature** to download the current file

To apply one note across the whole feature at once, use **Review every test case at once** at the top of the tab and click **Update all**. This runs as a single pass over all ticked scenarios, so the change lands consistently everywhere and the model can see the whole feature (it won't create a scenario that duplicates an existing one). If the note asks for coverage nothing currently provides, it may add a new scenario for it.

What the tick controls

A tick means "this is live". It does two things at once:

	Ticked	Unticked
Export	included in the `.feature` file	left out
Reviews (Update / Update all)	may be rewritten	reproduced exactly as-is, never changed

So to rework only part of a test case: untick the steps you're happy with, leave the rest ticked, write your note, and hit Update. Unticked steps stay in the document byte-for-byte — the server re-inserts any the model fails to reproduce, so excluded content can't be silently lost. `Update all` skips unticked scenarios entirely.

1. **Coverage tab** — every extracted requirement is listed with the scenario(s) that currently demonstrate it. Tick a requirement once you've manually confirmed it's genuinely covered, then **Save changes**.

- **Write the missing test cases** generates scenarios for any requirement nothing covers, without touching what already exists.
- **Export traceability matrix** produces a requirement <-> test case sheet with gaps flagged in red.

Scenario Outlines

Cucumber has two pipe-table constructs and they are **not** interchangeable:

	Runs	Placeholders	Use for
Scenario Outline + `Examples:`	Once per Examples row	Yes — `<name>` in steps	The same behaviour checked against several values
Data Table on a step	Once	No	One step handed a list of records

The generator now picks correctly — several cases differing only by input become one Outline, not a data table.

To parameterise a step by hand: select the word in the step text and press <>. It wraps the word in <>, converts the scenario to a Scenario Outline, and adds an `Examples:` column named after the word — with an empty row for you to fill in. Add more rows with **+ Row**; each row is one test run.

<> is the only per-step button. Data tables come from generation and stay fully editable, but there's no per-step button to add one by hand — <> covers the parameterising case, which is what you normally

want.

Renaming an `Examples:` column **rewrites every** `<placeholder>` **that uses it**, across step text and data-table cells, so the outline can't silently break. Column names must exactly match the placeholder names — the quality check reports it as an error if they don't:

```
Scenario Outline: Verify each main navigation link opens its section
  Given a visitor is on the homepage
  When they select the <navigation link> link
  Then the <section> section opens

  Examples:
    | navigation link | section      |
    | Home            | Introduction |
    | About           | About me     |
```

History and diffs

Every change — a manual edit, a review, a gap fill — snapshots the previous state first. Pick a version from **Compare with earlier version** to see exactly what changed: scenarios added or removed, titles renamed, steps added or dropped, and ticks flipped. Use it to check what a review pass actually did before you accept it. The last 50 versions per run are kept.

Exports

Button	File	Contains
Export .feature	`.feature`	Gherkin with type tags and aligned data tables — drops into a Cucumber repo
Export Excel	`.xlsx`	**Summary** (counts, type breakdown, gaps) · **Test Cases** (Test ID / Summary / Description / Test Steps / Priority / Test Type) · **Traceability**
Export traceability matrix	`.xlsx`	Traceability + Summary only

The Test Cases sheet uses the same column layout as the team's `scripts-to-excel` skill, so it drops into the existing QA tracker. Priority is inferred from the test type (happy path and security are High); adjust it in the sheet.

Runs are saved automatically on generation and persist across restarts — use the **Previous runs** dropdown in the header to reopen one.

How generation actually works

`server/lib/claudeCli.js` spawns `claude -p --output-format stream-json --system-prompt <QA-lead persona> --model sonnet --tools ""` and pipes the prompt in over **stdin** (never as a command-line argument, so pasted content can't be interpreted as shell syntax). `--tools ""` disables every built-in tool — the agent gets no file, shell, or network access of its own.

For a URL it additionally gets `--mcp-config mcp/playwright.json --strict-mcp-config` plus an explicit `--allowedTools` list naming only the browser tools it may use. It navigates to the page and takes an **accessibility snapshot** (real element roles, labels, and text — far more reliable than a screenshot or raw HTML, and it works on client-side-rendered SPAs). The browser runs headless with an isolated, throwaway profile, so it carries none of your cookies or logins.

The CLI's streamed NDJSON events are parsed to drive the Live activity panel, and the model's final JSON reply (the actual test plan) is parsed out of the result envelope.

Because this is a real Claude Code invocation, each generation/update consumes your normal Claude Code usage.

Covering a whole site, not just one page

Pasting a bare link (nothing else in the box) always goes through discovery first — **Generate test cases** is disabled for a link on its own, so you see the site's actual pages before committing to a run. Add a sentence of description alongside the link and direct generation opens back up, since that text becomes context either way. To cover a site:

1. Paste the site's URL and click **Explore the whole site first**.
2. It maps the site and lists the pages it found, each with what a user does there, and flags for **has a form** and **needs sign-in**.
3. Tick the pages you want. Pages with forms are pre-ticked, since those are usually the highest-value ones; **only pages with forms** / **all** / **none** re-select in one click.
4. Click **Generate for N selected pages**. It visits each in turn, then writes one suite covering them all.

Each scenario records which page it came from — shown as a badge on the card, and as a `# Page: <url>` comment in the exported `.feature` when a run spans more than one page.

How discovery works: a real headless browser crawls the site directly — no model call at all for the crawl itself, which is what makes it fast (a 12-page site typically maps in 15–25 seconds). It reads each page's title and inspects its DOM for forms and login walls, then one single cheap AI call writes a short description for each page from the titles alone. If that direct crawl can't find any real links to follow (a JS-only single-page app with no real `<a href>` tags, or the very first page it tries fails to load), it automatically falls back to the slower method — a full agentic pass where the model browses the site itself — so discovery still completes either way.

Notes:

- Crawling stays on the same site. External links (social, GitHub, docs) are dropped, and so are non-page assets (PDFs, images, stylesheets) that a link might point at.
- A plain anchor (`#contact`) counts as part of its page; a hash **route** (`#/active`) counts as a separate page, since in a single-page app those show different content.
- Discovery is strictly read-only — it never signs in, submits a form, or clicks anything, whatever the interaction setting says.
- Pages behind a login show up flagged but can't be explored until you've set up a **signed-in session**. On a site where everything is behind the login, discovery will honestly report just the login page.
- Default cap is 12 pages (25 max), to bound both time and usage.

Sharing a run with a colleague

You can hand a run to someone else and they can carry on reviewing, revising and exporting it in their own copy of the app. Two formats:

Export	Carries	Use when
Share file (JSON) — `*.testrun.json`	Everything: requirements, coverage ticks, test types, Scenario Outlines + Examples, data tables, and which steps are ticked	Handing work to a teammate — **prefer this**
Export .feature	Scenarios, steps, tables, outlines, and `@tags`	The file also has to go into a repo, Jira, or a Cucumber runner

To send: open the run, click **Share file (JSON)** (or **Export .feature**), and send them the file.

To receive: drop the file onto **"Someone sent you a file?"** at the top of the page, or click *Choose a file*. It becomes a new run — reviewable, revisable with **Update** / **Update all**, and re-exportable in any format. It never overwrites your existing runs.

A `.feature` file carries no requirements list, so the **Coverage** tab starts empty when you import one (the app tells you). Test types are recovered from `@tags`, so a file this app produced keeps them. That's why the JSON share file is the better handoff.

Imports are treated as untrusted input: 2 MB cap, bounded scenario/step/row counts, fresh ids assigned throughout, and `Background:` / `Rule:` blocks are flattened rather than dropped — with a note telling you what changed.

Pages behind a login

If you paste a URL that needs signing in — including **Google / SSO** — the agent on its own just lands on the login screen and writes test cases about *that*. Handing it credentials wouldn't fix it either: Google actively blocks automated browsers and refuses the sign-in outright.

So the agent never signs in. There are two ways to hand it a session you already have.

Option A — borrow the session from your own Chrome (recommended)

No second sign-in at all: you're already logged in somewhere, so lend it that. Under **Options -> Borrow your Chrome session**:

1. Open the site in Chrome, signed in. Press <kbd>F12</kbd> -> **Console**.
2. Click **Copy the snippet** in the app, paste it into the console, press Enter. It copies your session to the clipboard.
3. Paste that back into the app and click **Save session**.

The snippet only *reads* — it collects cookies and localStorage for that one origin and copies them out. Your password is never typed anywhere but your own browser, and nothing is sent to the model.

The one real limitation: JavaScript cannot read `HttpOnly` cookies. Apps that keep a token in `localStorage` (most modern SPAs) work fine; apps whose session lives in an HttpOnly cookie (most traditional server-rendered apps) will not be captured, and the app tells you so when you save. For those, either use Option B, or export a proper Playwright storage state and paste *that* — the app accepts it and it does include HttpOnly cookies:

```
npx playwright codegen --save-storage=state.json https://your-app.example.com
```

`sessionStorage` also can't be replayed — Playwright only restores cookies and localStorage. The app warns you if it finds a token there.

Option B — sign in through a Playwright browser

```
npm run login -- https://your-staging-app.example.com
```

Opens a real, visible browser (your installed Chrome if present). Sign in however you normally would — Google SSO, MFA, all of it — then press Enter in the terminal. Saved to `.browser-profile/`. This one *does* capture HttpOnly cookies, but some sites refuse a Playwright-driven browser.

Either way, tick **"Use my signed-in browser session"** before generating or exploring. A borrowed session takes precedence over a `npm run login` profile when both exist.

> **A saved session is a credential.** `.browser-profile/` and `data/storage-state.json` hold a live session for whatever you signed into — anyone who gets those files gets that session. - Use a **dedicated test account**, never your personal or work login. - It's gitignored, and you must **exclude it when zipping this project** for a teammate — they run `npm run login` with their own test account. - On a regulated project, treat it as a credential: same handling rules, and check with your security team before pointing it at anything real.

Interactive exploration

With the checkbox off (the default), the agent may only `navigate`, `snapshot`, read the console, and close the browser — it physically cannot click, type, or submit.

With it on, it additionally gets click/type/fill/select/press-key, so it can submit a form with bad input and capture the real error wording. The prompt forbids destructive actions (no payments or checkouts, no deleting data, no sending real messages, no persisting settings changes, no real credentials or personal data), but **a prompt is a guardrail, not a hard guarantee**. Only enable it against a staging or test environment you're authorized to exercise, never against production or anywhere a submitted form creates real records, charges, or notifications.

Running the tests

```
npm test
```

29 offline tests covering the parts where a regression would silently corrupt someone's test cases: step reconciliation (unticked steps must never be lost), the quality checks, Gherkin export formatting and escaping, and type resolution. No Claude calls, so it runs in under a second.

Limitations

- Generated test cases never reproduce a real person's name, email, phone number or social handle taken off a page — people are referred to by role ("the site owner", "the listed contact") and values are synthetic, even when the detail is the thing under test.
- Exploration reads what's on the page as loaded. The agent only ever signs in when you've explicitly given it a test username/password (see **Extra instructions**) — it never logs in with a credential it finds on the page itself. For a page behind a login you don't have test credentials for, see **Pages behind a login**.
- The fast crawler enumerates real `<a href>` links; it won't discover a page whose only route there is a JS click handler with no real link underneath (rare, but it happens). The agentic fallback usually can, since it reads the rendered page rather than just its links.
- The "explore first" gate is a UI nudge backed by a matching server check, not a hard security boundary — nothing stops a script calling the API directly with a single page in the `pages` array.
- Uploaded files are capped at 2 MB and only `.txt`/`.md`/`.pdf`/`.docx` are supported; a scanned PDF with no real text layer extracts nothing (and says so) rather than running OCR.
- If the model doesn't have enough to go on, it may write scenarios that assert generic behavior ("an error message is shown") rather than exact wording — that's intentional; the skill is instructed not to invent copy it hasn't actually seen.
- `Background` and `Rule` blocks aren't produced, and multiple `Examples:` blocks per outline aren't supported (one per scenario).
- Priority in the Excel export is inferred from test type, not judged per case.
- One generation at a time is the sane usage pattern; nothing stops two browser tabs from each starting a run, but each spawns its own CLI process and browser.
- Single-user by design: the server has no authentication and is meant to run on localhost. Share the folder, not a hosted instance.

Compliance note

Requirement text, URLs, and generated test cases are processed through your Claude Code session (Anthropic's models). **Do not paste real customer data, credentials, production PII/PHI, or internal URLs with regulated data-residency requirements — use synthetic/placeholder data instead** (this mirrors the rule the generator itself follows when writing example data). If this tool is used on projects with specific data-handling or vendor-approval requirements, confirm with your compliance team first.

Note that pointing it at a **URL** sends that page's rendered content to the model, not just the URL — so an internal page containing real records means that content leaves your machine. Prefer staging environments seeded with synthetic data. Guidance here is general: validate it against your own legal/compliance counsel before using this on a regulated project.

Every generate/save/update action is appended to `data/audit.log` (action, run id, timestamp) for traceability. Requirement and test case content is *not* written to the audit log — only to the per-run JSON files in `data/runs`.

Sharing this with others

This folder is self-contained — zip it up (excluding `node_modules/` and `data/`, per `.gitignore`) and hand it to a teammate. They run `npm install && npm start`, provided they also have Claude Code installed

and logged in on their own machine.